Geant4 User Guide

Francisco García-Infantes

April 28, 2026

Abstract

Throughout this User Guide, I will describe the code used for simulations involving DSSSD detectors. In Sec. 1, I briefly introduce the main components of the simulation code and explain how to run it. In Sec. 2, I go through the `UserInput.inp` file, describing all available options and parameters. In Sec. 3, I detail the structure and content of the output file. In Sec. 4, I provide information about the geometry and explain how to implement the sample for your specific case, which is the only part that you are expected to modify yourself in the code. Finally, in Sec. 5, I describe additional analysis tools (e.g. coincidence analysis).

As a final note, if you have ideas for improvements or encounter any issues, please let me know so they can be implemented or fixed in the main code.

1 Introduction

Geant4 is an open-source software toolkit used to simulate how particles (such as electrons, photons, or protons) travel through and interact with matter.

If you look inside the source tree of Geant4, it is organized in a fairly standard C++ library layout, but at a large scale because it covers many physics domains.

1. **include/** — headers (interfaces) This folder contains header files (.hh / .h) that define the public API:
   - Class declarations (e.g. G4Track, G4ParticleDefinition)
   - Function prototypes
   - Constants and enums
2. **src/** — implementations This folder contains the source files (.cc) that implement the functionality declared in include/:
   - Implementations of classes declared in include/
   - Physics processes (e.g. scattering, decay)
   - Geometry handling logic
3. **Module-based structure** Geant4 is split into many modules, each with its own include/ and src/ directories. For example:
   - geometry/ → detector shapes, volumes
   - particles/ → particle definitions
   - processes/ → physics interactions
   - tracking/ → how particles move step-by-step

This is a very brief overview. If you want more information about how Geant4 works, check the following links:

- Geant4 manual: `https://geant4.web.cern.ch/docs/` (see User Guides for application developers)
- Courses and tutorials: `https://geant4.web.cern.ch/docs/tutorials`

- Basic examples: `https://geant4.web.cern.ch/documentation/pipelines/master/bfad_html/ForApplicationDevelopers/Examples/BasicCodes.html`

  (This can also be found in the Geant4 installation directory.)

It is not mandatory to consult these links, as everything has been implemented in order to allow running simulations without deep knowledge of Geant4 internals beyond basic geometry. However, I encourage everyone to explore them in depth to fully understand what Geant4 is doing and what you are getting from the simulations.

For our codes, once you download them and to run them, go to the /code folder and:

- (If a /build folder already exists) *rm -rf build*
- *mkdir build* & *cd build*
- *cmake ..*
- *make -j8*
- *cp ../vis.mac .*

Once built, the executable ***myDSSSD*** should appear. To run the code, there are three options available:

1. *./myDSSSD -i UserInput.inp* This is the basic option where you pass the input file to Geant4 for simulation. This is mandatory.
2. *./myDSSSD -i UserInput.inp -t 2* This enables multithreading. You use more than one CPU core to run the simulation; the number of threads depends on your machine. This speeds up the simulation.
3. *./myDSSSD -i UserInput.inp -r 271683* This allows you to set a specific random seed for reproducible simulations.
4. *./myDSSSD -i UserInput.inp -t 2 -r 271683* All options can be combined.

2 User Input Parameters

To explain the User Input I will divide in different sections.

2.1 Sample Features

The sample is defined in `Construction.cc`, as explained in Sec. 4. In the `UserInput` file, you must define the emission point of the primary particle, which must correspond to the sample definition. Therefore, the same geometry definition must be consistent between `UserInput` and `Construction.cc`.

In the `UserInput`, you define the shape, dimensions, and rotation of the sample. This is directly related to the emission point of the primary particle.

The parameters required to define the sample features are:

- **name**. This must be identical to the name defined in `Construction.cc`, as explained in Sec. 4.
- **SampleShape**. Two options are available:
  1. **Circular**
     - **diameter (mm)**: Diameter of the sample. This must match the value defined in `Construction.cc`.
     - **thickness (mm)**: Thickness of the sample. This must match the value defined in `Construction.cc`.
  2. **Rectangular**
     - **height (mm)**: Height of the sample. This must match the value defined in `Construction.cc`.
     - **width (mm)**: Width of the sample. This must match the value defined in `Construction.cc`.
     - **thickness (mm)**: Thickness of the sample. This must match the value defined in `Construction.cc`.

These parameters affect the emission point of the primary particle; however, they do not modify the physical dimensions of the sample defined in `Construction.cc`. It is essential that the parameters are consistent between `UserInput.inp` and `Construction.cc`.

In addition, a sample rotation feature has been implemented. This affects both `UserInput.inp` and `Construction.cc`. When defining the sample, ensure that the rotation is specified correctly, as described in Sec. 4.

The available rotation options are:

- **RotationAngleX (degrees)**: Implemented, but in principle not required for our case.
- **RotationAngleY (degrees)**: Rotation around the Y-axis; this is relevant for our case.
- **RotationAngleZ (degrees)**: Implemented, but in principle not required for our case.

An example of the circular sample with and without rotation is shown in Fig. 1.

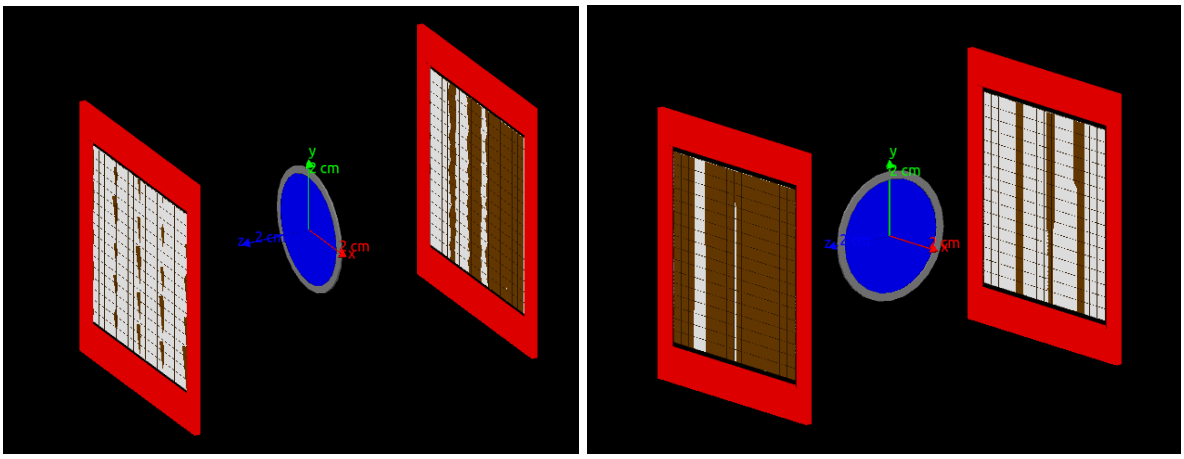

Figure 1: Left panel: Circular sample w/o rotation. Right panel: Circular sample rotated 90 degrees.

2.2 Particle Information

Once the emission point/volume is defined, the next step is to specify the properties of the simulated particles. First, we define the particle type and its energy. In addition, we must define the masses of the incident particle and target nucleus, together with the mass of the ejected particle and the residual nucleus.

- **particle**: Defines the particle type (e.g. proton, triton, alpha, etc.). If you want to simulate a different particle, refer to the Geant4 manual.
- **energy**: Defines the energy of the incident particle (e.g. neutron beam). It is important to define this value because the energy of the ejected particle is calculated within the code.
- **energyspread**: Defines the energy spread of the incident particle.
- **incidentparticle**: Mass of the incident particle in Da.
- **targetnucleus**: Mass of the target nucleus in Da.
- **ejectedparticle**: Mass of the ejected particle in Da.
- **residualnucleus**: Mass of the residual nucleus in Da.

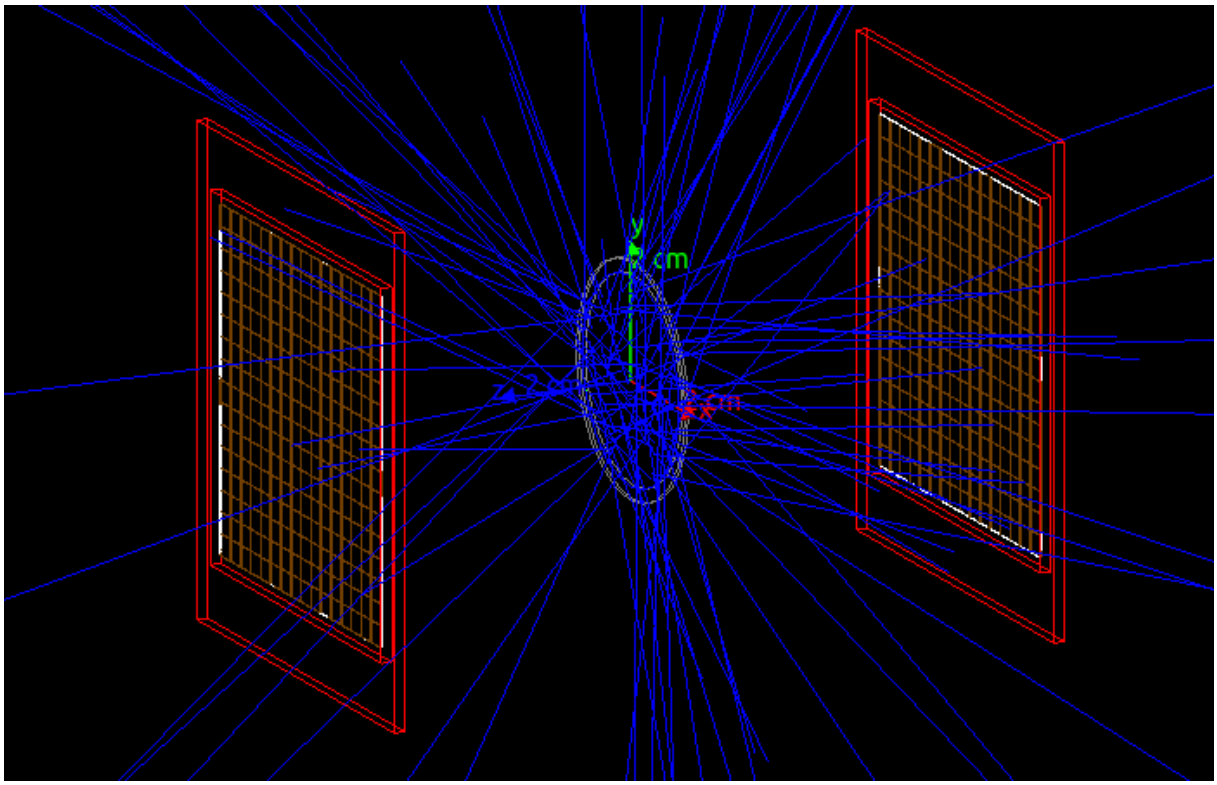

Figure 2: Circular sample and 2×DSSSD with 100 particles simulated.

2.3 Reference Frame

To correctly account for the reference frame, we start from the center-of-mass frame and transform it to the laboratory frame. Three reference frame options are implemented; however, the third option is not fully available yet and is only briefly described here.

- **ReferenceFrame**: Three options are available.
  1. **LabFramefromCMFrame**: This option transforms from the center-of-mass frame to the laboratory frame. As mentioned before, the kinetic energy of the ejected particle is calculated based on the masses provided and the energy of the incident particle.
  2. **LabFramefromCMFrame_Simultaneous**: This option works as the previous one but simulates two particles in the same event. For this option, an additional parameter must be provided:
     - **secondparticle**: Defines the secondary particle (e.g. alpha in the ^{6}Li(n,α)T reaction).
  3. **LabFrame**: This option is available only for the simulation of decay sources.

Advice: Read *Nuclear Physics of Stars* by Christian Iliadis (Appendix C) or *Fundamentals of Nuclear Physics* by Arya Atam Parkesh (pages 80–92). It is important to perform the calculations yourself in order to fully understand the concepts and the motivation behind this implementation in the simulations.

2.4 Distribution Simulation

There are two implementations to account for the particle distribution: angular distribution and spatial (profile) distribution of the incident particles. Instructions on how to obtain the inputs for the angular and profile distributions can be found in Appendix AAA.

We start with the angular distribution:

- **angulardistribution**: Two options are available.

1. **Isotropic**: This option assumes an isotropic distribution in the center-of-mass frame. It does not account for reaction anisotropy.
2. **Anisotropic**: This option accounts for reaction anisotropy. The anisotropy must be provided as input, typically obtained from TALYS.
   - **AngularDistributionFile**: File containing the anisotropy distribution obtained from TALYS.

On the other hand, for the spatial (profile) distribution, three options are available. The third one is related to a decay source, is not fully implemented, and is not required for n_TOF simulations.

- **emission**: Three options are available.
  1. **H**: Homogeneous option. This does not account for the spatial profile of the incident particle beam.
  2. **B**: This option accounts for the spatial profile of the incident particle beam. Three parameters must be provided:
     - **bifFilename**: File containing the beam profile. The input file for the simulation is the output file from the transport code. The minimum stepping for the transport code simulation must be at least 0.1 cm.
     - **MissalignX (mm)**: Shift of the profile center along the X-axis.
     - **MissalignY (mm)**: Shift of the profile center along the Y-axis. These parameters allow the study of the influence of beam–target misalignment in the X–Y plane.
  3. **Decay**: This option accounts for the spatial distribution of a radioactive source. The following parameters must be provided:
     - **Source**: Two values in the same row must be provided: the mass number and atomic number of the radioactive source.
     - **MissalignX (mm)**: Shift of the source center along the X-axis.
     - **MissalignY (mm)**: Shift of the source center along the Y-axis.
     - **MissalignZ (mm)**: Shift of the source center along the Z-axis. ***Not implemented yet.***

2.5 Detector Features

The following parameters are related to the detectors implemented in the simulation. Two types of detectors are available: double-sided silicon strip detectors (DSSSD) and single-sided silicon strip detectors (SSD). For each detector, you can define the thickness and its position relative to the sample. The geometrical dimensions of the detector (e.g. height, width, etc.) are fixed by default, based on standard silicon detector geometries.

The parameters that must be provided are:

- **NumberDetectors**: Number of detectors to be built in the simulation. Once the number is defined, each detector must be specified individually as DSSSD or SSD.
  - **DSSSD**: A double-sided silicon strip detector. The number corresponds to the detector ID.
  - **SSD**: A single-sided silicon strip detector. The number corresponds to the detector ID. IDs must be sequential (e.g. 1, 2, 3, ...) and not skipped (e.g. 1, 3, 4 is not allowed).
- **Thickness (mm)**: Thickness of each detector in mm.
- **z (cm)**: Distance from the center of the sample to the center of the detector.
- **theta (deg)**: Polar angle used to place the detector.
- **phi (deg)**: Azimuthal angle used to place the detector.
- **TranslationX (cm)**: Shift of the detector in the X-axis after applying the z, theta, and phi positioning.
- **TranslationY (cm)**: Shift of the detector in the Y-axis after applying the z, theta, and phi positioning.

- **TranslationZ (cm)**: Shift of the detector in the Z-axis after applying the z, theta, and phi positioning. The last three parameters allow the study of possible misalignment between the sample and the detectors.

All parameters must be provided for each detector, even if their value is zero.

2.6 Extra Parameters

A few additional parameters must be defined in the `UserInput` file:

- **Nevents**: Number of events to be simulated.
- **verbose**: Tracking information flag. This controls the amount of information printed during the simulation depending on the value provided. Refer to the Geant4 manual for details.
- **visualize**: Enables geometry visualization when set to true, and disables it when set to false. It is important to verify the geometry after implementing the sample and detectors.
- **outputFilename**: Name of the output file. Provide the full path of the directory together with the file name.

3 Output File

The output file provides several trees and histograms to analyze the simulation results. Depending on the simulation settings, additional histograms may be available. By default, the output contains two main trees and a set of histograms determined by the input parameters.

The trees generated in the simulation are:

- **DSSSD**: This tree contains all the information about the detected events. The parameters included are:
  1. **EventN**: Event number.
  2. **det**: Detector ID where the event was recorded.
  3. **strip**: Strip ID where the event was recorded.
  4. **Side**: Detector side identifier:
     - Side = 0: Event recorded in an SSD.
     - Side = 1: Event recorded on the front side of a DSSSD.
     - Side = 2: Event recorded on the back side of a DSSSD.
  5. **edep**: Energy deposited by the detected particle.
  6. **particle**: Particle identification number. This includes both single-particle and coincidence events (the latter for the *LabFramefromCMFrame_Simultaneous* option):
     - 1: Proton
     - 2: Triton
     - 3: Proton + Triton coincidence
     - 4: Alpha particle
     - 5: Proton + Alpha coincidence
     - 6: Triton + Alpha coincidence
- **Scattering_Angle**: This tree contains angular information of the emitted particle in both reference frames:
  - **EventN**: Event number.
  - **Theta_Lab**: Emission angle in the laboratory frame (degrees).
  - **Theta_CM**: Emission angle in the center-of-mass frame (degrees).
- **Scattering_Angle_Secondary**: This tree contains angular information for the secondary particle (only for the *LabFramefromCMFrame_Simultaneous* option):

- **EventN**: Event number.
- **Theta_Lab**: Emission angle in the laboratory frame (degrees).
- **Theta_CM**: Emission angle in the center-of-mass frame (degrees).

The most relevant tree for analysis is **DSSSD**, as it contains the full detector response and allows detailed post-simulation studies.

In addition, several histograms are produced to provide a quick overview of the simulation results:

- **Energy deposition histograms**:
  - **SSD_DetNum**: Energy spectrum of particles detected in each SSD.
  - **DSSSD_Front_DetNum**: Energy spectrum for the front side of each DSSSD.
  - **DSSSD_Back_DetNum**: Energy spectrum for the back side of each DSSSD.

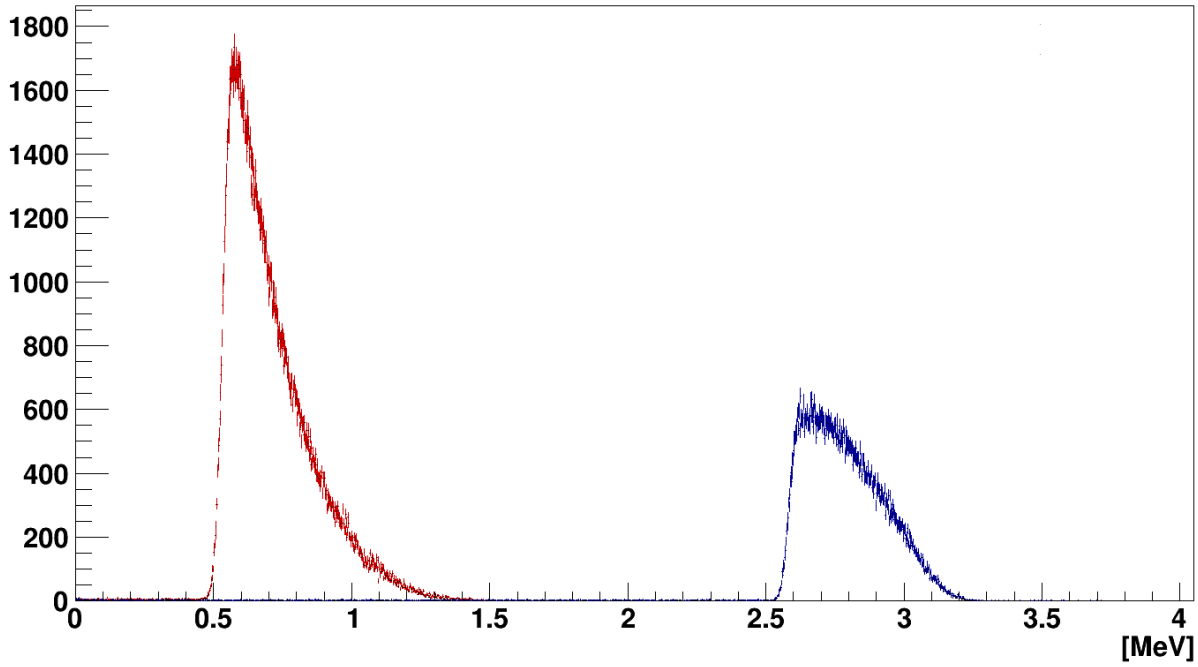

Figure 3: Deposited energy in the silicon detectors. The red spectrum corresponds to the deposited energy in the SSD, while the blue spectrum corresponds to the deposited energy in the DSSSD. The simulation was performed assuming isotropic emission, homogeneous emission, and a circular sample.

- **Angular histograms**:
  - **Initial angle from the lab frame SSD_DetNum**: Emission angles of particles detected in each SSD.
  - **Initial angle from the lab frame DSSSD_DetNum**: Emission angles of particles detected in each DSSSD.

- **Particle emission position histograms**:
  - **hXParticlePosition**: X-coordinate of the emission point.
  - **hYParticlePosition**: Y-coordinate of the emission point.
  - **hZParticlePosition**: Z-coordinate of the emission point.
  - **hXParticlePositionDetected**: X-coordinate of detected events.
  - **hYParticlePositionDetected**: Y-coordinate of detected events.
  - **hZParticlePositionDetected**: Z-coordinate of detected events.

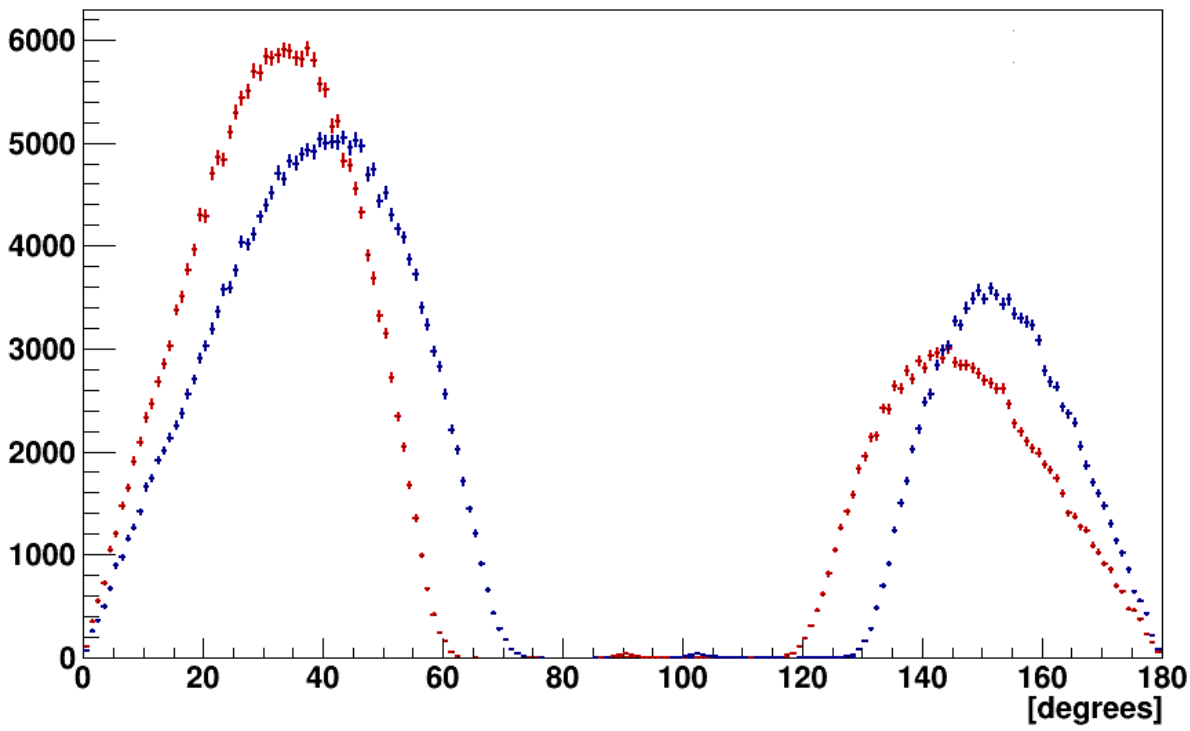

Figure 4: Emission angle of particles detected in the silicon detectors. The red distribution corresponds to the angle in the laboratory frame, while the blue distribution corresponds to the angle in the center-of-mass frame. The simulation was performed assuming isotropic emission, homogeneous emission, and a circular sample.

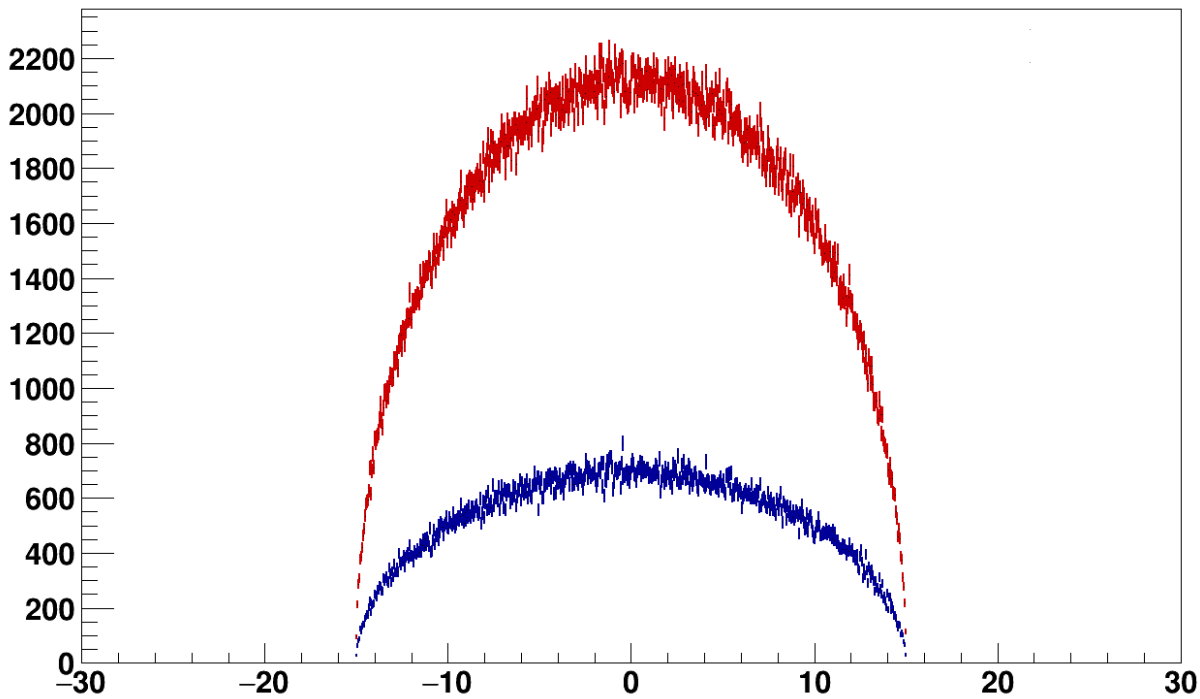

Figure 5: Position along the X-axis of the emitted particles. The red distribution corresponds to all emitted particles, while the blue distribution represents the emission points of the detected particles. The simulation was performed assuming isotropic emission, homogeneous emission, and a circular sample.

- **2D histograms**:
  - **hXYParticlePosition**: XY projection of the emission points.
  - **hXZParticlePosition**: XZ projection of the emission points.
  - **hYZParticlePosition**: YZ projection of the emission points.
  - **Lab Angle Emission Simultaneous**: Correlation between center-of-mass and laboratory frame angles for simultaneous emission events.

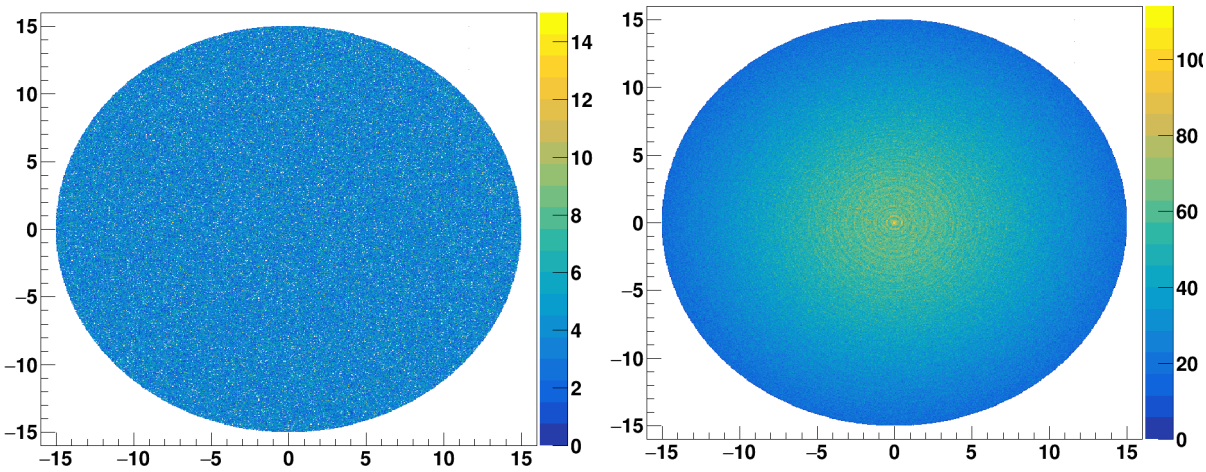

Figure 6: Left panel: Emission points in the XY plane for a homogeneous distribution. Right panel: Emission points in the XY plane for a profile distribution.

4 Geometry. Sample definition

To implement the samples some information is provide in this section. The first step is to define the material in the code. All the implementations have to be done in the construction.cc and construction.hh. As mentioned above, the dimension of the sample should match with the definition in the UserInput.inp to emit the particle from the same volume.

The material definition is shown in Fig. 7.

```
void MyDetectorConstruction::LoadMaterials() {
    // BEGIN MATERIALS LIST
    G4NistManager* materials_list = G4NistManager::Instance();  // Materials database

    air = materials_list->FindOrBuildMaterial("G4_AIR");
    O  = materials_list->FindOrBuildMaterial("G4_O");
    Al = materials_list->FindOrBuildMaterial("G4_Al");
    F = materials_list->FindOrBuildMaterial("G4_F");
    Ca = materials_list->FindOrBuildMaterial("G4_Ca");
    Cl = materials_list->FindOrBuildMaterial("G4_Cl");
    Na = materials_list->FindOrBuildMaterial("G4_Na");
    Si = materials_list->FindOrBuildMaterial("G4_Si");
    Li = materials_list->FindOrBuildMaterial("G4_Li");
    B = materials_list->FindOrBuildMaterial("G4_B");
    Au = materials_list->FindOrBuildMaterial("G4_Au");
    quartz = materials_list->FindOrBuildMaterial("G4_SILICON_DIOXIDE");
    mylar = materials_list->FindOrBuildMaterial("G4_MYLAR");

    //Sodium Chloride target
    cl35 = new G4Material("NaCl", 2.16*g/cm3, 2);
    cl35->AddMaterial(Na, 0.5);
    cl35->AddMaterial(Cl, 0.5);
```

Figure 7: Material definition in GEANT4.

The isotopes for your sample have to be define first. Then, you have to define the sample as:

- ***cl35 = new G4Material("NaCl", 2.16*g/cm3, 2);*** The first is the name, the second is the mass in g/cm^3 and the las value are the number of isotopes.
- ***cl35→AddMaterial(Na, 0.5);*** You have to add the number of isotopes one by one giving the isotopic ratio.

In the sample above you define for the NaCl 1 Na isotope and 1 for the Cl. The material has to be implemented in the ***LoadMatrials*** function.

Then, you have to define the geometry to build your sample. In Fig. 8 an example is shown for the LiF sample. The LiF layer has been defined with the mylar backing. In adittion, the Al ring has been defined.

```
void MyDetectorConstruction::ConstructTarget_LiF(G4LogicalVolume* mother) {
    //Standards definition
    targetrot = new G4RotationMatrix();
    G4float TargetRotationx, TargetRotationy, TargetRotationz;
    userInp->Return_Rotation(TargetRotationx, TargetRotationy, TargetRotationz);
    targetrot->rotateX(TargetRotationx * deg);
    targetrot->rotateY(-TargetRotationy * deg);
    targetrot->rotateZ(TargetRotationz * deg);
    targetDiameter = 30 * mm;
    targetThickness = 0.001 * mm;
    targetRadius = targetDiameter/2.0;

    G4double mylarthickness = 0.005 * mm;

    //mylar foil
    //Gold layer definition
    G4Tubs* mylarfoil = new G4Tubs("Mylar_foil", 0, targetRadius, mylarthickness / 2.0, 0*degree, 360*degree);
    mylarlayer = new G4LogicalVolume(mylarfoil, mylar, "Mylar_foil");
    new G4PVPlacement(0, G4ThreeVector(0, 0, mylarthickness / 2 + targetThickness / 2.0), mylarlayer, "Mylar_foil", mother, false, 0, flagOverlaps);
    mylarlayer->SetVisAttributes(G4Colour::Blue());

    //Sodiumd Chloride target
    Target_LiF = new G4Tubs("Target_LiF",
                            0,
                            targetRadius,
                            targetThickness/2.0,
                            0*degree,
                            360*degree);
    targetLiF = new G4LogicalVolume(Target_LiF, LiF, "Target_LiF");
    new G4PVPlacement(targetrot, G4ThreeVector(0, 0, 0), targetLiF, "Target_LiF", mother, false, 0, flagOverlaps);
    targetLiF->SetVisAttributes(new G4VisAttributes(G4Colour::Green()));

    // Aluminum Ring
    G4double Althickness = 0.5 * mm;
    solidAlRing = new G4Tubs("AlRing", targetRadius, targetRadius + 2 * mm, Althickness / 2, 0, 360 * deg);
    G4LogicalVolume* logicAlRing = new G4LogicalVolume(solidAlRing, Al, "AlRing");
    new G4PVPlacement(targetrot, G4ThreeVector(0., 0., -1. * Althickness / 2 + targetThickness / 2), logicAlRing, "AlRing",
                      mother, false, 0, flagOverlaps);
    logicAlRing->SetVisAttributes(G4Colour::Gray());
}
```

Figure 8: Sample definition in GEANT4.

The lines you have to include are shown in Fig. 9. This lines allow to rotate the sample from the UserInput.inp.

```
targetrot = new G4RotationMatrix();
G4float TargetRotationx, TargetRotationy, TargetRotationz;
userInp->Return_Rotation(TargetRotationx, TargetRotationy, TargetRotationz);
targetrot->rotateX(TargetRotationx * deg);
targetrot->rotateY(-TargetRotationy * deg);
targetrot->rotateZ(TargetRotationz * deg);
```

Figure 9: Rotation parameters to be included in each sample definition.

Then, when you define the placement of your sample you have to include the Target Rotation definition like:

- ***targetrot = new G4RotationMatrix();*** Definition of the rotation.
- ***new G4PVPlacement(0, G4ThreeVector(0, 0, mylarthickness / 2 + targetThickness / 2.0), mylarlayer, "Mylar_foil", mother, false, 0, flagOverlaps);*** Definition of the rotation in the sample definition.

The last step, you have to include your sample in the loop to be able to build it, as shown in Fig. 10. You have to include another **else if** loop and to give a name to your sample. For the Chloride sample the name is **NaCl**, which is the same one in the UserInput.inp.

```
targetrot = new G4RotationMatrix();
G4float TargetRotationx, TargetRotationy, TargetRotationz;
userInp->Return_Rotation(TargetRotationx, TargetRotationy, TargetRotationz);
targetrot->rotateX(TargetRotationx * deg);
targetrot->rotateY(-TargetRotationy * deg);
targetrot->rotateZ(TargetRotationz * deg);
```

Figure 10: Rotation parameters to be included in each sample definition.

This has to be include in the construction.cc, but some definition has to be done in the construction.hh. These are:

- ***void ConstructTarget_NaCl(G4LogicalVolume* mother);*** You have to define the target constructor function in your header.
- ***G4Material *cl35*** You have to define the material name.

You can include other definitions but are not needed. If you have any doubt, you can check the following links: `https://geant4.web.cern.ch/documentation/pipelines/master/bfad_html/ForApplicationDevelopers/Detector/Geometry/geometry.html`.

5 Analysis Codes

Two extra codes have been included. One is dedicated for MONNET and the second one is for n_TOF. Both of them are very basic codes, which doesn't do a lot of analysis. They show the deposited energy when there are coincidence between the detectors and w/o coincidence between detectors, to be able to do a bit of analysis in terms of coincidences.

5.1 Analysis Code MONNET

An additional analysis code has been developed for MONNET, named `GEANT4_Analysis_MONNET.cpp`. This code only requires the name of the output file generated by the Geant4 simulation and reads the tree produced during the simulation. At the moment, it is implemented for the specific case of the *LabFramefromCMFrame_Simultaneous* option and for a detector setup consisting of one forward and one backward detector. An example is shown in Fig. 11.

In addition to the energy spectra, the number of events detected per strip is also stored in the output ROOT file generated by the analysis code.

The simulations were performed using a circular sample without rotation, with two DSSSDs placed at 0° and 180° and a distance of 2.5 cm. The goal was to reproduce the MONNET experiment (note that the exact experimental geometry is not yet implemented). The angular distribution was taken into account in the simulations.

5.2 Analysis Code n_TOF

An additional analysis code has been developed for n_TOF, named `GEANT4_Analysis_n_TOF.cpp`. This code only requires the name of the output file generated by the Geant4 simulation and reads the tree produced during the simulation. At the moment, it is implemented for the specific case of the *LabFramefromCMFrame* option and for a detector setup consisting of one single-sided detector (SSD) and one double-sided detector (DSSSD), with the DSSSD placed behind the SSD. An example is shown in Fig. 12. The incident particle energy was 3 MeV.

In addition to the energy spectra, the number of events detected per strip is also stored in the output ROOT file generated by the analysis code.

The simulations were performed using a circular sample without rotation, with one SSD and one DSSSD placed at 45°, at distances of 2.5 cm and 3.5 cm, respectively. The goal was to reproduce the n_TOF experiment (note that the exact experimental geometry is not yet implemented). Both angular and spatial (profile) distributions were taken into account in the simulations. The incident particle energy was 3 MeV.

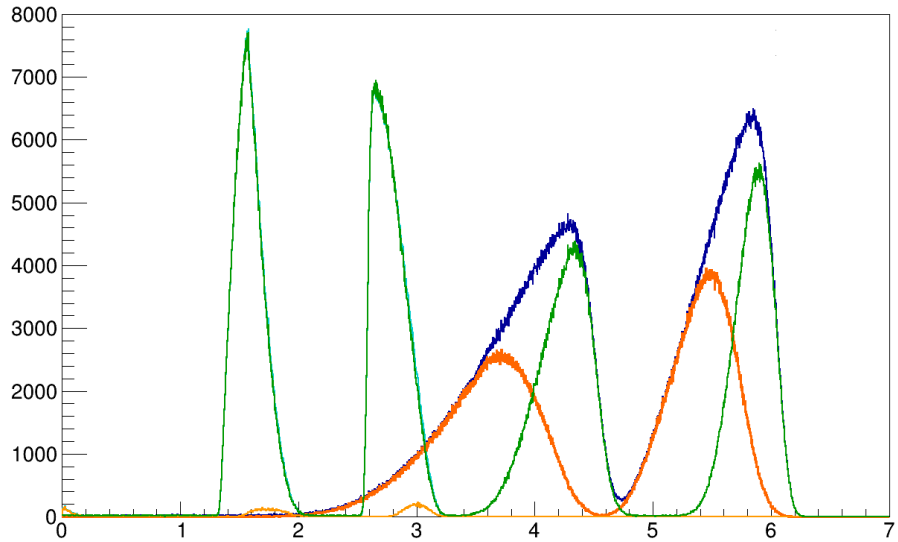

Figure 11: Disentanglement of the deposited energy. Coincidence events between both DSSSDs are shown in dark green; the total deposited energy in the forward detectors is shown in dark blue, and in the backward detector in light blue. Events detected only in the forward detector are shown in dark orange, while those detected only in the backward detector are shown in light orange.

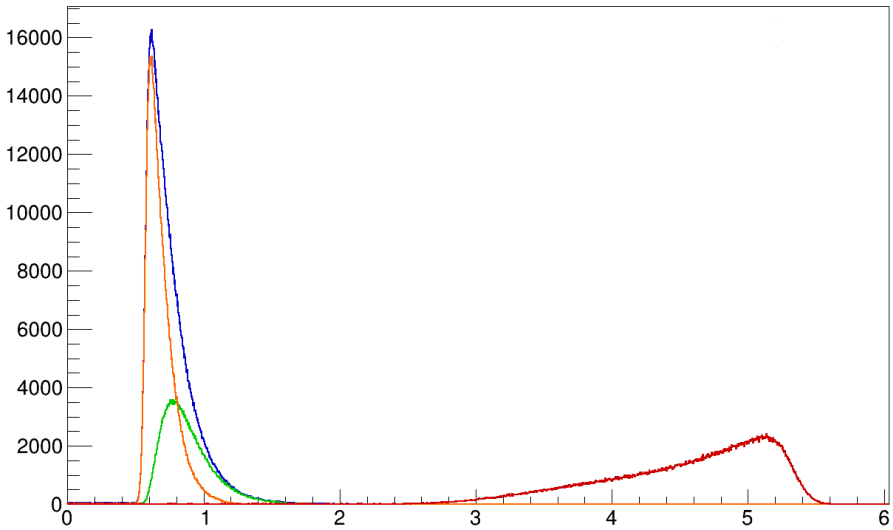

Figure 12: Disentanglement of the deposited energy. The dark blue curve shows the total deposited energy of the simulation; the dark red curve shows events detected in the backward detector; the dark green curve shows events detected only in the forward detector; and the orange curve shows events detected in coincidence with the backward detector.

Appendix

A Output files for Profile

The code reads directly the output file from the transport code (TC). To run the TC, you must access `lxplus` and execute:

`python3 /eos/experiment/ntof/simul/transport_GUI/GUI/App.py`

You must then provide the correct input parameters in the TC. More information about the transport code can be found in the following documentation: `https://twiki.cern.ch/twiki/pub/NTOF/TransportCode/TC_GUI_user_manual.pdf`

It is important to select an appropriate *step size for resampling*, which must be 0.01 or smaller.

In addition, the flight path entered in the TC is not identical to the one used in the analysis. Further details can be found in: `https://indico.cern.ch/event/1168514/contributions/5152775/attachments/2566369/4425858/Status%20of%20the%20FLUKA%20simulations-6.pdf` (see slide 26). In practice, the correct value to use is the nominal flight path minus 33.08 cm.

Once the simulations are completed, the output files can be averaged, producing both a `.txt` file and a `.root` file. For the Geant4 analysis, the `.txt` file is required.

B Output files for Angular Distribution

To include the anisotropy emission, you have to get the output from talys. To install talys use this link: `https://nds.iaea.org/talys/`. When you download, follow the manual to install it, and to run it. In the repository, there is an example for the ^{6}Li(n,α)T. There are some output files. Depending of the ejected particle (e.g. triton) you have to get that output file for the simulations. No more steps neeeded.